MARKET BASKET ANALYSIS: POTENTIAL COMBOS AND CUSTOMER PSYCHOLOGY

1. Introduction

A cosmetics and personal care retailer wants to optimize revenue by understanding customer shopping behavior. Instead of focusing solely on selling individual products, the business wants to identify groups of items that customers frequently purchase together in a single order to implement cross-selling campaigns and rearrange its store layout.

2. Problem statement

- Which combo has the strongest linking power? Are there any specific product groups included?
- Why are these product groups often purchased together with each other?

3. Data used

- EcomSales.csv: Sales transaction data.
- Product.csv: Product information.

I. Setup & Data Ingestion

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
#
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
```

```
pd.set_option('display.max_colwidth', None)
pd.set_option('display.expand_frame_repr', None)
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
df_sales = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/EcomSales.csv')
df_sales.head()
```

```
df_products = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/Product.csv')
df_products.head()
```

II. Data Cleaning & Pre-processing

```
# Declare a function to check the dataframe.

def check_df(df):
  print('=' * 50)
  print('DATAFRAME INFORMATION')
  print('=' * 50)
  # Print concise summary of the DataFrame
  df.info()

  print('=' * 50)
  print('NULL VALUES')
  print('=' * 50)
  # Count null values per column
  print(df.isna().sum())

  print('=' * 50)
  print('DUPLICATE VALUES')
  print('=' * 50)
  # Count duplicate rows
```

```
print(f" Number of duplicates: {df.duplicated().sum()}")

print('=' * 50)
print('DESCRIPTIVE STATISTICS')
print('=' * 50)
# Generate descriptive statistics
print(df.describe().T)
```

Inspect df_sales

```
check_df(df_sales)
```

```
# Create a box plot to visualize the distribution and outliers of 'Quantity'
sns.boxplot(data= df_sales, x= 'Quantity')
```

```
# Create a box plot to visualize the distribution and outliers of 'Sales'
sns.boxplot(data= df_sales, x= 'Sales')
```

Orders with revenue identified as outliers are likely orders from potential customers, a minority who require more special attention than the majority.

High revenue also means a large number of products in a single order (in the `Quantity` column), and `Profit` also fluctuates significantly, leading to the result of becoming outliers.

Outliers in the AnnualIncome of df_customers may include potential customers.

```
# Check for unusual codes in df_sales

import re

# Define the pattern for ProductCode (1 uppercase letter followed by 6 digits, e.g., P000001)
pattern = r'^[A-Z]\d{6}$'
```

```
# Find data that does not conform to the above format
non_conforming_product_codes = df_sales[~df_sales['ProductCode'].str.fullmatch(pattern, na=False)]

print(non_conforming_product_codes) # Display non-conforming product codes
```

```
# Define the pattern for RegionCode (1 uppercase letter followed by 4 digits, e.g., R0001)
pattern = r'^[A-Z]\d{4}$'

# Find data that does not conform to the above format
non_conforming_region_codes = df_sales[~df_sales['RegionCode'].str.fullmatch(pattern, na=False)]

print(non_conforming_region_codes) # Display non-conforming region codes
```

For data with different encoding formats than those in the dimension table, it is necessary to check with relevant sources or confirm with stakeholders to verify before making any modifications to these unusual product and region codes.

Inspect df_products

```
check_df(df_products)
```

III. MBA

Arguing for appropriate thresholds for each indicator across 12,316 orders.

Support is set at a minimum of 0.2%, meaning product pair A and B must appear together in at least 25 orders. This level is just enough to eliminate randomness (noise), yet low enough to capture potentially profitable niche cross-selling combinations that no one else has noticed.

Set the minimum Confidence to 20% to ensure a large enough customer base for product A to make the cross-sell campaign effective in terms of revenue.

Method 1: Self-join

```
df = df_sales.merge(df_products, on='ProductCode', how='inner')
df.head()
```

```
df_pre_mba = df[['OrderID', 'Subcategory']]
df_pre_mba.head()
```

```
df_mba = df_pre_mba.merge(df_pre_mba, on = 'OrderID', how = 'inner', suffixes= ('_A','_B'))
df_mba = df_mba[df_mba['Subcategory_A'] < df_mba['Subcategory_B']]
df_mba.head()
```

```
# Support(A-->B) = No of orders have A & B divided to Total orders

df_freq = df_mba.groupby(['Subcategory_A', 'Subcategory_B'], as_index = False).agg(Total_orders_A_B = ('OrderID','nunic
df_freq['Total_orders'] = df_mba['OrderID'].nunique()
df_freq['Support'] = df_freq['Total_orders_A_B'] / df_freq['Total_orders']

df_freq.head(10)
```

```
df_freq_A = df_mba.groupby(['Subcategory_A'], as_index = False).agg(Total_orders_A = ('OrderID','nunique'))
df_freq_B = df_mba.groupby(['Subcategory_B'], as_index = False).agg(Total_orders_B = ('OrderID','nunique'))
```

```
# Confidence(A-->B) = No of orders have A & B divided to No of orders have B

df_freq = df_freq.merge(df_freq_A, on = 'Subcategory_A', how = 'inner')
df_freq['Confidence'] = df_freq['Total_orders_A_B'] / df_freq['Total_orders_A']
```

```
df_freq.head(10)
```

```
# Lift(A-->B) = Confidence(A-->B) / Support(A)

df_freq = df_freq.merge(df_freq_B, on = 'Subcategory_B', how = 'inner')
df_freq['Lift'] = df_freq['Confidence'] / (df_freq['Total_orders_B'] / df_freq['Total_orders'])

df_freq.head(10)
```

```
df_freq.describe().T
```

```
# Setting thresholds for each indicator
df_filtered = df_freq[(df_freq['Support'] >= 0.002) & (df_freq['Confidence'] >= 0.20)].sort_values(by = 'Lift', ascendi
df_filtered.head(10)
```

```
# Select top N combinations for visualization to keep it readable
top_n = 10
df_plot = df_filtered.head(top_n)

# Create a combined label for the product combinations
df_plot['Combination'] = df_plot['Subcategory_A'] + ' -> ' + df_plot['Subcategory_B']

plt.figure(figsize=(10, 7))
sns.barplot(x='Lift', y='Combination', data=df_plot, palette='viridis')
plt.title(f'Top {top_n} Subcategory Combinations', fontsize=16)
plt.xlabel('Lift', fontsize=12)
plt.ylabel('', fontsize=12)
plt.tight_layout()
plt.show()
```

Method 2: Implementing Apriori algorithm

```
# Importing Required Libraries
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

import warnings

# Ignore all warnings of the DeprecationWarning type.
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

```
# Loading data
df_mlx = pd.merge(df_sales[['OrderID', 'ProductCode']],
              df_products[['ProductCode', 'Product', 'Subcategory']],
              on='ProductCode', how='inner')

df_mlx.head()
```

```
# Group Subcategories by OrderID
basket = df.groupby('OrderID')['Subcategory'].apply(list).reset_index()
transactions = basket['Subcategory'].tolist()

transactions
```

```
# Convert to One-Hot Format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)

df_encoded.head()
```

```
# Apply a minimum support threshold of 0.2%
frequent_subcategorysets = apriori(df_encoded, min_support=0.002, use_colnames=True)
print("Total Frequent Subcategorysets:", frequent_subcategorysets.shape[0])
print("Total Frequent Subcategorysets:", frequent_subcategorysets)
```

```
# Set threshold for Confidence
rules = association_rules(frequent_subcategorysets, metric="confidence", min_threshold=0.2)


# Set Lift > 1
rules = rules[rules['antecedents'].apply(lambda x: len(x) >= 1) & rules['consequents'].apply(lambda x: len(x) >= 1)]
print("Association Rules:", rules.shape[0])
```

```
rules['antecedents_str'] = rules['antecedents'].apply(lambda x: ', '.join(list(x)))
rules['consequents_str'] = rules['consequents'].apply(lambda x: ', '.join(list(x)))
rules['Combo'] = rules['antecedents_str'] + " + " + rules['consequents_str']

# Top 10 combinations
top_10_rules = rules.sort_values('lift', ascending=False).head(20)
```

```
# Visualize

plt.figure(figsize=(12, 6))
sns.barplot(data=top_10_rules, x='lift', y= 'Combo', palette='magma')
plt.title('Top 10 Subcategory Combinations', fontsize=16)
plt.xlabel('Lift', fontsize=12)
plt.ylabel('', fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.6)
# plt.tight_layout()
plt.show()
```

```
df_products['Subcategory'].unique()
```

IV. Conclusion

4.1. In method 1

The chart reveals that the association rule of Shampoos and conditioners -> Vitamins and supplements has an exceptionally high Lift score (over 5.0), standing out significantly from the rest of the data. This indicates that when a customer purchases shampoo or conditioner, there is a remarkably high probability that they will also buy vitamins.

The "Nail care products" category also frequently appears as a cross-purchased item alongside primary products (accounting for 4 out of 10 combinations).

Modern consumers are not only concerned with physical appearance but also prioritize inner health. When purchasing external personal care products, they are driven by a "beauty from within" mindset, leading them to buy additional vitamins or dietary supplements for internal care. This reflects a consumer behavior oriented towards holistic solutions.

When customers are in the mindset of shopping for makeup, they desire cohesive perfection down to the smallest detail. Furthermore, nail care products are typically compact and inexpensive, making them highly susceptible to impulse buying.

The combination of purchasing shampoo/conditioner alongside hair dye clearly reflects the mindset of DIY hair coloring: the fear of hair damage and color fading drives customers to buy specialized shampoos/conditioners for recovery and color retention.

Additionally, the combination of perfume and hair dye reveals a customer segment with a strong desire for personal reinvention (possibly for a holiday, special event, or simply a mood shift). A major change in physical appearance (hair color) is accompanied by a change in personal style (fragrance).

Based on the above analysis, here are actionable strategies that can be implemented immediately:

- Strategy 1: Boost Cross-Selling. Implement "Frequently Bought Together" recommendations on the website/app. As soon as a customer adds Shampoo/Conditioner or Hand Cream to their cart, trigger a mandatory pop-up suggesting Vitamin products with a message like: "Nourish from within." Apply the same logic to the Nail Care category.
- Strategy 2: Launch Themed Product Bundles. For example, introduce a "Hair Savior Bundle" combining Hair Dye + Color-Protecting Shampoo/Conditioner + Hair Serum, or a "Total Glow" combo featuring Moisturizing Hand Cream + a bottle of Vitamin C/Collagen.
- Strategy 3: Set up Automated Email/Zalo Workflows. If a customer has recently purchased Hair Dye but hasn't bought a specialized Shampoo/Conditioner, send an automated message within 48 hours: "Loving the bold new hair color! Protect it and make it last longer with our specialized shampoo line - Here is a 15% discount code just for you."

4.2. In method 2

The combination yielding the highest Lift score (over 1.75) is the grouping of candles, sprays, diffusers + bath oils, bubbles and soaks + face masks and exfoliators. It is clearly evident that this represents an at-home spa and beauty product category.

The "Self-Care" & "Self-Reward" Mentality: This combination indicates that customers are not just purchasing product functionality; they are "buying a relaxing evening." When a person decides to buy bath oils and scented candles, they have already planned a dedicated relaxation time for themselves. Therefore, purchasing an additional face mask or exfoliator is a completely logical psychological next step to complete that "At-Home Spa" experience.

Based on this insight, here are a few ideas for business campaigns:

- Create Product Bundles & Gift Sets: Design and immediately launch gift sets named "Weekend Spa Kit" or "Relaxation Bundle". Apply a 10-15% discount for the bundle compared to buying the items individually, and release these promotions during holidays throughout the year.
- Visual Merchandising & UI/UX: Optimize the recommendation algorithm on the website/app. When a customer adds an essential oil diffuser to their cart, a "Frequently Bought Together" pop-up must immediately display premium face masks or exfoliators. In physical stores, create "Experience Corners" by placing compact, travel-sized masks right next to the scented candles and body wash shelves to stimulate impulse buying.
- Emotion-Driven Communication Campaigns: Shift the focus of your content marketing. Instead of advertising the individual features of a scented candle or a face mask, produce short, ASMR-style videos depicting a tired person coming home from work, stepping into a candlelit bathroom, soaking in the tub, and applying a face mask. Selling the "relaxation story" will organically drive sales for the entire product combination.

Additionally, further exploration can be done on combinations involving accessories and makeup tools (brushes and applicators), which also exhibit a high Lift score of over 1.5. Analyzing these combinations will help uncover deeper customer intentions when purchasing these specific product categories together.